

09-Network Communication(网络通信)

鸣谢：黑马程序员



💡 Tip

Previous Chapter(上一章): 08-Multithreading(多线程)

- [🔗Markdown](#)
- [🔗PDF](#)
- [🔗HTML](#)

Next Chapter(下一章): 10-Reflection(反射)

- [🔗Markdown](#)
- [🔗PDF](#)
- [🔗HTML](#)

• 09-Network Communication(网络通信)

一、概述

二、前置知识

1.基本通信架构

2.网络通信三要素

2.1 IP地址：设备在网络中的唯一标识

P1 IP地址的两种形式

P2 域名(Domain Name)

P3 公网和内网IP

P4 特殊IP

P5 常用的终端IP命令

2.2 端口号：应用程序在设备中的唯一标识

P1 端口分类

P2 注意事项

2.3 通信协议：设备连接和数据在网络中传输的规则

P1 UDP协议

P2 TCP协议

P1 通过三次握手建立可靠连接

P2 通过四次挥手断开连接

三、网络编程

1. `InetAddress` 类：代表IP地址

2.UDP通信

2.1 `DatagramSocket` 类：创建客户端和服务端

P1 构造器

P2 常用方法

2.2 `DatagramPacket`：创建数据包

P1 构造器

P2 常用方法

2.3 一发一收

2.4 多发多收

3.TCP通信

一、概述

- 网络编程可以让设备中的程序和网络上其他设备中的程序进行网络通信，实现数据交互。
- `java.net` 包下提供了网络编程的解决方案。

二、前置知识

1.基本通信架构

- **CS架构**：Client-Server（客户端-服务器）
- **BS架构**：Browser-Server（浏览器-服务器）

2.网络通信三要素

2.1 IP地址：设备在网络中的唯一标识

P1 IP地址的两种形式

- **IPv4**：32bits(4B)，一般采用点分十进制表示。如192.168.1.66。
- **IPv6**：
 - 128bits(16B)，一般采用冒分十六进制表示法，即分成8段表示，每段有16bits，其中每4bits转成一个十六进制数表示，段之间用冒号隔开。如2001:0DB8:0000:0000:0008:0800:200C:417A。
 - 为了书写方便，IPv6还提供了压缩格式，具体压缩规则为：每组中的前导0都可以省略。因此上述地址可写为：2001:0DB8:0:0:8:800:200C:417A
 - 如果存在连续两个及以上均为0的段，可以用双冒号来代替，因此上述地址可进一步简写为：2001:0D88::8:800:200C:417A。
 - 需要注意的是，在一个IPv6地址中只能使用一次双冒号，否则当计算机将压缩地址恢复成完整地址时，无法确定每个双冒号表示几个0。

P2 域名(Domain Name)

IP地址是数字形式，可读性差且难以记忆，因此人们发明了可读性较高且容易记忆的域名。

当用户想要访问某个网站时，他在计算机设备上输入域名，本地**DNS(Domain Name System)**服务器就会将域名解析成对应的IP地址并返回给设备，此时设备就能通过网站的IP地址访问网站。（注：此处没有考虑DNS缓存不命中的情况）

P3 公网和内网IP

- **公网IP**：可以连接互联网的IP地址。
- **内网IP**：也叫局域网IP，只能在组织或机构内部使用。192.168.开头的IP是常见的局域网IP。

P4 特殊IP

- 127.0.0.1，别名 `localhost`，代表本机IP，只会寻找当前所在的主机。

P5 常用的终端IP命令

- 查看本机IP地址：`ipconfig(Windows)/ifconfig(Linux)`。
- 检查本机与目标IP之间的网络是否连通：`ping 目标IP`。

2.2 端口号：应用程序在设备中的唯一标识

16bits，表示范围是0~65535。

P1 端口分类

- **周知端口**：0~1023，被预先定义的知名应用所占用，比如HTTP占用80端口，FTP占用21端口。
- **注册端口**：1024~49151，分配给用户进程或某些应用程序，比如MySQL默认占用3306端口，Tomcat默认占用8080端口。
- **动态端口**：49152~65535，一般不固定分配给某种进程，而是采取动态分配的策略。

P2 注意事项

我们自己开发的程序一般使用注册端口，且同一设备中不能出现两个端口号相同的程序，否则会出错。

2.3 通信协议：设备连接和数据在网络中传输的规则

OSI网络参考模型	TCP/IP网络模型	各层对应	面向操作
应用层	应用层	HTTP、FTP、SMTP...	应用程序需要关注的：浏览器，邮箱。 程序员一般在这一层开发
表示层			
会话层			
传输层	传输层	UDP、TCP...	选择使用的TCP，UDP协议
网络层	网络层	IP...	封装源和目标IP
数据链路层	数据链路层+ 物理	比特流...	物理设备中传输
物理层			

P1 UDP协议

 Important

UDP(User Datagram Protocol)：用户数据报协议。

- 特点：无连接、不可靠传输。
 - 不事先建立连接，数据以“包”为单位进行传输，“包”的数据部分一般包含：源IP、源程序端口号、目的IP、目的程序端口号、传输数据（64KB以内）等。
 - 发送方发送前不会确认接收方是否在线，数据在传输过程中丢失也不管，接收方收到数据后也不返回确认包ACK，因此传输不可靠。
- 但是UDP通信效率高，适用于语音通话、视频直播等对通信可靠性要求不高的场景。

P2 TCP协议

📢 Important

TCP(Transmission Control Protocol): 传输控制协议。

- **特点**: 面向连接、可靠传输。
- 但是TCP通信效率相对不高。
- TCP采用三个步骤实现可靠传输:
 1. 通过**三次握手**建立可靠连接;
 2. 传输数据后进行确认, 如果客户端没有收到服务器的确认或确认信息有误, 则客户端必须重发;
 3. 通过**四次挥手**断开连接。

P1 通过三次握手建立可靠连接

💡 Tip

可靠连接: 通信双方收发信息正常无问题 (全双工通信)。

SYN(Synchronize Sequence Numbers): 同步序列号, 是TCP协议头部的一个控制标志位, 用于发起连接请求并同步初始序列号。

- 当SYN=1且ACK=0时, 表示纯连接请求;
- 当SYN=1且ACK=1时, 表示对连接请求的确认。

第一次握手: 客户端向服务器发送 `SYN=1, ACK=0` 的报文, 携带随机生成的初始序列号 ISN, 请求与服务器建立连接。

第二次握手: 服务器收到客户端发来的连接请求报文后, 回复 `SYN=1, ACK=1` 的报文, 确认客户端的连接请求并发送自己的ISN。

第三次握手: 客户端收到服务器发来的确认报文后, 回复 `SYN=0, ACK=1` 的报文, 确认服务器的ISN, 连接建立完毕。

确认事项	第一次握手后，服务器得知	第二次握手后，客户端得知	第三次握手后，服务器得知
客户端发送能力正常	✓		
服务器接收能力正常		✓	
服务器发送能力正常		✓	
客户端接收能力正常			✓

P2 通过四次挥手断开连接

- 第一次挥手：**主动关闭方向被动关闭方发送断开连接请求。
- 第二次挥手：**被动关闭方返回一个响应：稍等。
- 第三次挥手：**被动关闭方完成剩余数据传输后，返回一个响应：确认断开连接。
- 第四次挥手：**主动关闭方向被动关闭方发送报文：收到剩余数据，正式确认断开连接。

三、网络编程

1. `InetAddress` 类：代表IP地址

序号	方法	说明
01	<code>static InetAddress getLocalHost()</code>	将本机IP封装成一个 <code>InetAddress</code> 对象并返回。
02	<code>static InetAddress getByName(String host)</code>	根据IP或域名，返回一个 <code>InetAddress</code> 对象。
03	<code>String getHostName()</code>	获取该 <code>InetAddress</code> 对象的主机名（一般是域名）。
04	<code>String getHostAddress()</code>	获取该 <code>InetAddress</code> 对象的IP地址信息。
05	<code>boolean isReachable(int timeout)</code>	在指定时间（单位：毫秒）内，判断主机与该IP对应的主机之间是否连通。

2.UDP通信

2.1 `DatagramSocket` 类：创建客户端和服务端

P1 构造器

构造器	说明
<code>public DatagramSocket()</code>	创建 客户端 的 <code>Socket</code> 对象，系统会为其随机分配一个端口号。
<code>public DatagramSocket(int port)</code>	创建 服务端 的 <code>Socket</code> 对象，并指定端口号。

P2 常用方法

方法	说明
<code>void send(DatagramPacket dp)</code>	发送数据包。
<code>void receive(DatagramPacket dp)</code>	使用数据包接收数据。

2.2 `DatagramPacket`：创建数据包

P1 构造器

构造器	说明
<code>public DatagramPacket(byte[] buf, int length, InetAddress, int port)</code>	创建用于发送数据的数据包对象，必须指定服务端IP和端口号。
<code>public Datagram(byte[] buf, int length)</code>	创建用于接收数据的数据包对象。

P2 常用方法

方法	说明
<code>int getLength()</code>	获取当前数据包实际接收到的字节数。
<code>InetAddress getAddress()</code>	获取发送当前数据包的客户端的IP地址对象。
<code>int getPort()</code>	获取发送当前数据包的客户端端口。

2.3 一发一收

⚠ Caution

用完 `Socket` 后记得关闭资源，建议使用 `try-with-resource` 的代码方式进行自动关闭。

- `Server`:

```
1 package udp;
2
3 import java.io.IOException;
4 import java.net.DatagramPacket;
5 import java.net.DatagramSocket;
6 import java.net.InetAddress;
7 import java.net.UnknownHostException;
8
9 public class Server {
10     public static final InetAddress IP;
11     public static final int PORT = 6666;
12     public static final int LENGTH = 1024 * 64; // 64KB
13
14     static {
15         try {
16             IP = InetAddress.getLocalHost();
17         } catch (UnknownHostException e) {
18             throw new RuntimeException(e);
19         }
20     }
21
22     public static void main(String[] args) {
23         System.out.println("Server starts");
24         try (
25             DatagramSocket socket = new DatagramSocket(PORT); //
用完自动关闭资源
26         ) {
27             byte[] buf = new byte[LENGTH];
28             DatagramPacket packet = new DatagramPacket(buf,
LENGTH);
29
30             socket.receive(packet);
31             System.out.println("Server receives a packet from
Client, the contents are as follows:");
```

```

32         System.out.println("Client IP: " +
packet.getAddress().getHostAddress());
33         System.out.println("Client Port: " + packet.getPort());
34         int length = packet.getLength();
35         System.out.println(new String(buf, 0, length));// 接收多
少，就倒出多少
36     } catch (IOException e) {
37         throw new RuntimeException(e);
38     }
39 }
40 }

```

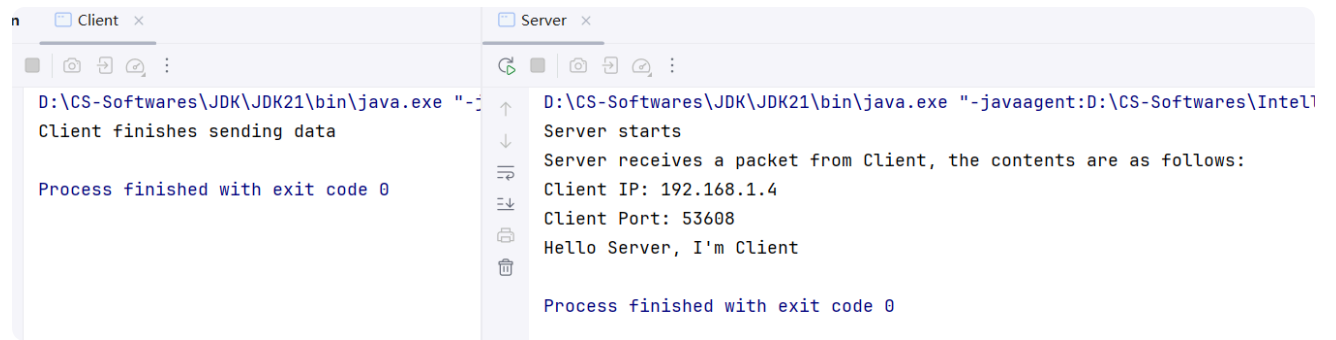
- **Client:**

```

1  package udp;
2
3  import java.io.IOException;
4  import java.net.*;
5
6  public class Client {
7      public static void main(String[] args) {
8
9          try (
10              DatagramSocket socket = new DatagramSocket();// 用
完后自动关闭资源
11          ) {
12              byte[] buf = "Hello Server, I'm Client".getBytes();
13              DatagramPacket packet = new DatagramPacket(
14                  buf, buf.length, Server.IP, Server.PORT);
15
16              socket.send(packet);
17              System.out.println("Client finishes sending data");
18          } catch (IOException e) {
19              throw new RuntimeException();
20          }
21      }
22  }

```

- 先启动服务端，再启动客户端，控制台输出如下：



```
n Client x
D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\CS-Softwares\Intel\jbr\bin\javaagent.jar"
Client finishes sending data

Process finished with exit code 0

Server x
D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagent:D:\CS-Softwares\Intel\jbr\bin\javaagent.jar"
Server starts
Server receives a packet from Client, the contents are as follows:
Client IP: 192.168.1.4
Client Port: 53608
Hello Server, I'm Client

Process finished with exit code 0
```

2.4 多发多收

3.TCP通信